

CONCEPTOS DE BASES DE DATOS

CLASE 3



- **Organización de un archivo**
 - Al usar archivos con **registros de longitud fija**, cada registro ocupa el mismo tamaño. Sin embargo, los datos realmente no ocuparán la totalidad del espacio reservado → **ineficiencia en el uso del espacio**.
 - Se reserva espacio para el **peor caso**, aunque la mayoría de los registros no lo necesite.
 - Además, cuanto **mayor es la variabilidad** en el tamaño de los datos, **mayor es el desperdicio** en registros de longitud fija.

- **Organización de un archivo**

Ej: **persona = record**
nyap: string[50];
dir: string[50];
dni: string[12];
edad: integer;
obs: string[200];
end

Campo	Tamaño res.	Uso promedio
Nyap	50	20
Dir	50	28
Dni	12	11
Edad	2	2
Obs	200	59
Total	314 (*)	120 bytes

(*) En realidad, los strings en Pascal ocupan un byte adicional para almacenar la longitud efectiva, por lo que el tamaño del registro sería de 318 bytes. Sin embargo, para esta comparación este detalle no resulta relevante.

- **Organización de un archivo**
 - En el ejemplo dado, **se desperdicia mas del 60% del espacio** reservado para el registro.
 - Si el archivo tuviera **200.000 registros**, se desperdicia **38 MB** aproximadamente.
 - Impacto de la gran cantidad de espacio sin utilizar:
 - Tamaño del archivo.
 - Uso de memoria/disco.
 - Performance general.

- **Organización de un archivo**
 - La alternativa es utilizar archivos con **registros de longitud variable**, en donde **cada registro ocupa solo el espacio necesario**.
 - De esta forma, **se elimina el desperdicio** y se hace un **uso más eficiente** del almacenamiento.
 - Existen **diferentes opciones** para implementar archivos de este tipo.

- **Organización de un archivo**
 - **Características de estructura**
 - **Secuencia de bytes**
 - Unidad de lectura/escritura → **byte**
 - Difícil determinar el comienzo y el final de cada dato.
 - **Campos**
 - Unidad de lectura/escritura → **campo**
 - Permite separar cada dato
 - **Registros**
 - Unidad de lectura/escritura → **registro**
 - Es un conjunto de campos agrupados que **definen un elemento del archivo**

- **Campos**

- **Longitud predecible**

- Longitud fija: se rellena el campo → desperdicio de espacio.

- **Indicador de longitud**

- Valor al principio de cada campo.

- **Delimitador**

- Se usa una marca al final de cada campo, que es un carácter especial no utilizado como dato.

- **Ej: lectura y presentación de un archivo con delimitadores para los campos**

Datos del archivo: 3,14|2,71|0,577|1,618|42,0 <EOF>

Salida esperada:

1 3,14

2 2,71

3 0,577

4 1,618

5 42,0

- Ej: lectura y presentación de un archivo con delimitadores para los campos

PROGRAM leerArchivoCampos

CONST

delimitador = '|'

BEGIN

ABRIR archivo (como lectura/escritura)

nroCampos = 0

longCampo = leerCampo(archivo, contenidoCampo)

MIENTRAS (longCampo > 0)

INC nroCampos

ESCRIBIR_PANTALLA nroCampos

ESCRIBIR_PANTALLA contenidoCampo

 longCampo = leerCampo(archivo, contenidoCampo)

FIN MIENTRAS

CERRAR archivo

END

- **Ej: lectura y presentación de un archivo con delimitadores para los campos**

```
FUNCTION leerCampo(var archivo, var contenidoCampo)
BEGIN
    long = 0
    car = ""
    MIENTRAS(NOT EOF(archivo) & car <> delimitador )
        LEER_UN_CARACTER(archivo, car)
        SI (car <> delimitador )
            contenidoCampo[long] = car
            INC long
    FIN MIENTRAS
    DEVOLVER(long)
END
```

- **Registros**

- **Longitud predecible**

- Longitud fija: relleno de c/ campo → desperdicio de espacio.
- Cantidad de bytes: nro. fijo de bytes.
- Cantidad de campos: nro. fijo de campos.

- **Indicador de longitud**

- Se indica al inicio de cada registro.

- **Delimitador**

- Se usa una marca al final de cada registro (carácter especial no utilizado como dato y diferente de otros delimitadores).

- **Segundo archivo**

- Mantiene la dirección del byte de inicio de cada registro.

Archivos

Organización

- Ej: archivo con indicador de longitud (registros) y delimitadores (campos)

PROGRAM leerArchivoRegistros

CONST delimitador = '|'

BEGIN

ABRIR archivo

tomarReg(archivo, buffer, longRegistro) *{traslada un registro a un buffer}*

 posBuscada = 0

MIENTRAS (longRegistro > 0)

tomarCampo(buffer, longRegistro, posBuscada, campo) *{lee un campo}*

MIENTRAS (posBuscada > 0)

ESCRIBIR_PANTALLA campo

tomarCampo(buffer, longRegistro, posBuscada, campo)

FIN MIENTRAS

tomarReg(archivo, buffer, longRegistro)

FIN MIENTRAS

CERRAR archivo

END

Archivos

Organización

- Ej: archivo con indicador de longitud (registros) y delimitadores (campos)

PROCEDURE tomarReg (*var* archivo, *var* buffer, *var* long_reg)

BEGIN

SI EOF(archivo) ENTONCES

long_reg = 0

SINO

LEER *long_reg* {nro entero que indica la cantidad de bytes del registro}

LEER *long_reg* bytes del archivo en *buffer*

END

Archivos

Organización

- Ej: archivo con indicador de longitud (registros) y delimitadores (campos)

PROCEDURE tomarCampo(buffer, long_reg, var pos_bus, var campo)

BEGIN

SI (*pos_bus* == *long_reg*) ENTONCES

pos_bus = 0

SINO

LEER byte en la *pos_bus* de *buffer*

MIENTRAS(*pos_bus* < *long_reg* & byte <> delimitador)

COLOCAR byte en *campo*

INC(*pos_bus*)

LEER byte en la *pos_bus* de *buffer*

FIN MIENTRAS

END

- **Pascal**

- En Pascal es posible aplicar este tipo de organizaciones definiendo una variable como **archivo sin tipo** (file).
- Esta elección de archivo permite realizar la transferencia **carácter a carácter** (byte a byte).
- De esta forma es posible el uso de archivos con **registros de longitud variable**.

- **Pascal**

- Registros de longitud variable:
 - Espacio de almacenamiento → optimizado.
 - Lectura/escritura registros → implementación ad-hoc.
- Una organización posible:
 - Secuencia de caracteres (archivo sin tipo).
 - Delimitadores de fin de campo ('#') y fin de registro ('@').
 - Marca EOF.

- **Ej: creación de un archivo de empleados**

```
program ejemplo_4_5;  
Var  
    empleados: file; {archivo sin tipo}  
    nombre,apellido,direccion,documento: string;  
  
Begin  
    Assign(empleados,'empleados.txt');  
    Rewrite(empleados,1);  
    writeln('Ingrese el Apellido');  
    readln(apellido);  
    writeln('Ingrese el Nombre');  
    readln(nombre);  
    writeln('Ingrese direccion');  
    readln(direccion);  
    writeln('Ingrese el documento');  
    readln(documento);
```

Archivos

Organización

```
while apellido<>'zzz' do
  Begin
    BlockWrite(empleados,apellido,length(apellido)+1);
    BlockWrite(empleados,'#',1);
    BlockWrite(empleados,nombre,length(nombre)+1);
    BlockWrite(empleados,'#',1);
    BlockWrite(empleados,direccion,length(direccion)+1);
    BlockWrite(empleados,'#',1);
    BlockWrite(empleados,documento,length(documento)+1);
    BlockWrite(empleados,'@',1);
    writeln('Ingrese el Apellido');
    readln(apellido);
    writeln('Ingrese el Nombre');
    readln(nombre);
    writeln('Ingrese direccion');
    readln(direccion);
    writeln('Ingrese el documento');
    readln(documento);
  end;
close(empleados);
end.
```

- **Ej: visualización del archivo de empleados**

```
program ejemplo_4_6;  
Var  
    empleados: file; {archivo sin tipo}  
    campo, buffer :string;  
Begin  
    Assign(empleados,'empleados.txt');  
    reset(empleados,1);  
    while not eof(empleados) do  
        Begin  
            BlockRead(empleados,buffer,1);
```

Archivos

Organización

```
while (buffer<>'@') and not eof(empleados)do
begin
  while ((buffer<>'@') and (buffer<>'#') and
    not eof(empleados))do
  begin
    campo := campo + buffer ;
    BlockRead(empleados,buffer,1);
  end;
  writeln(campo);
  campo := '';
  if (buffer<>'@') then
    BlockRead(empleados,buffer,1);
end;

if not eof(empleados) then
  BlockRead(empleados,buffer,1);
end;
close(empleados);
end.
```


- **Clave de un registro**

En muchos casos, **no es necesario procesar todo el archivo**, sino acceder a **registros particulares**.

Para ello, cada registro posee uno o más campos que permiten identificarlo.

- Una **clave** es un conjunto de uno o más campos del registro que permiten **identificarlo** y **facilitar su recuperación**.

- **Clave de un registro**

Existen dos tipos de clave:

- **Clave primaria**

- Identifica de **forma única a cada registro** del archivo.
- No puede repetirse.

- **Clave secundaria**

- Puede identificar a **uno o más registros**.
- Se utiliza para búsquedas que no sean únicas.

- **Clave de un registro**

Se denomina **Forma Canónica** de una clave a una **representación normalizada de la clave**, obtenida a partir de reglas predefinidas.

El objetivo es garantizar que:

- Una misma clave tenga **siempre la misma representación**.
- Se **eviten errores** en búsquedas y comparaciones.

Ejemplo de normalización:

- Convertir a mayúsculas + Eliminar espacios + Quitar caracteres especiales.
- "Perez, Juan " → "**PEREZJUAN**"

- **Acceso directo**

Este modo de acceso permite **posicionarse directamente en el lugar de un registro**. Resulta **eficiente** cuando se necesita acceder a uno o pocos **registros específicos**.

- En archivos de **registros de longitud fija** es posible porque todos los registros ocupan lo mismo. (por ej. en Pascal implementado con la operación **seek**).
- En archivos de **registros de longitud variable** no es posible, ya que no se conoce el tamaño de cada registro.

- **Acceso directo**

Que los archivos de registros de longitud fija permitan acceso directo, **no significa que es posible encontrar un registro directamente por su clave.**

- Es necesario **conocer el lugar de comienzo del registro** requerido.
- En archivos de registros de longitud fija, cada registro tiene asociado un número llamado **NRR** (Número Relativo de Registro).
- El NRR es la **posición lógica del registro dentro del archivo.**

- **Acceso directo**

¿Cuándo no es posible aplicar acceso directo?

- Si no se conoce el NRR del registro buscado.
- Si se trabaja con archivos de registros de longitud variable.

Ante estos dos casos, y si además el archivo **no se encuentra ordenado**, la única alternativa es realizar una **búsqueda secuencial**.

Archivos

Acceso a los datos

- Ej: búsqueda secuencial de un empleado usando su clave

PROGRAM BuscarEmpleado

BEGIN

ABRIR archivo

LEER apellidoBuscado

LEER nombreBuscado

{A partir de los datos leídos desde teclado → construyo la clave en Forma Canónica}

claveBuscada = **CONSTRUIR_FC** (apellidoBuscado, nombreBuscado)

encontro = false

tomarRegistro(archivo, buffer, longRegistro)

Archivos

Acceso a los datos

- **Ej: búsqueda secuencial de un empleado usando su clave**

MIENTRAS (NOT encontro AND longRegistro > 0)

posBuscada = 0

tomarCampo(buffer, longRegistro, posBuscada, apellido)

tomarCampo(buffer, longRegistro, posBuscada, nombre)

{A partir de los datos leídos desde archivo → construyo la clave en Forma Canónica}

claveRegistro = **CONSTRUIR_FC** (apellido, nombre)

SI (claveRegistro == claveBuscada) **ENTONCES**

encontro = true

SINO **tomarRegistro**(archivo, buffer, longRegistro)

FIN MIENTRAS

SI (encontro) **ENTONCES** **MOSTRAR_REGISTRO**();

CERRAR archivo

END

Archivos

Mantenimiento

- Los procesos de algorítmica clásica que se examinaron en la clase anterior utilizan archivos con registros de **longitud fija**.
 - Agregar registros.
 - Modificar registros.
- Aún resta analizar:
 - Esos procesos en archivos con registros de **longitud variable**.
 - El proceso de **eliminación de registros**.

El impacto de las operaciones sobre un archivo depende del tipo de archivo y de la **frecuencia con la que se modifica su información**. Los tipos de archivo (según sus cambios) son:

- **Estáticos**
 - Presentan pocos cambios.
 - Pueden actualizarse mediante procesamiento por lotes.
 - No requieren estructuras adicionales para optimizar modificaciones.
- **Volátiles**
 - Presentan operaciones frecuentes (altas, bajas y modificaciones).
 - Su organización debe permitir cambios eficientes.
 - Requieren estructuras adicionales para mejorar los tiempos de acceso.

- Inserción de un registro
 - Registro de **longitud fija** → sin problemas
 - Registro de **longitud variable** → sin problemas
- Modificación de un registro
 - Registro de **longitud fija**
 - Dado que el tamaño del registro es siempre el mismo → sin problemas
 - Registro de **longitud variable**
 - Si ahora el registro tiene tamaño menor → sobra lugar
 - Si ahora el registro tiene tamaño mayor → no cabe

- Modificación de un registro

¿Qué se hace cuando el archivo tiene registros de **longitud variable**, pero el registro actualizado tiene un **tamaño mayor** y ya no cabe en el mismo lugar?

- **Opción 1:** agregar los datos adicionales al final del archivo.
 - Se utiliza un vínculo al registro original.
 - Hace **más complejo el procesamiento** del registro.
- **Opción 2:** eliminar e insertar el registro actualizado al final del archivo.
 - **Más simple** que la opción anterior.
 - Queda un **espacio vacío** (desperdiciado) en el lugar origen.

- Modificación de un registro
 - Entre las dos alternativas presentadas, se adopta la **reubicación del registro** como estrategia principal para la modificación.
 - Esta opción simplifica el acceso a los datos, aunque **genera espacios libres** en el archivo.
 - El enfoque se centrará entonces en los métodos de **eliminación de registros** y la **recuperación del espacio liberado**.

Archivos

Eliminación

- Eliminación de un registro
 - **Baja física**
 - El registro eliminado deja de estar físicamente en el archivo.
 - **Baja lógica**
 - El registro eliminado sigue estando en el archivo, pero marcado como eliminado.

- **Baja Física** → Compactación
 - Forma más simple: copiar todo a excepción de los registros eliminados.
 - Frecuencia (depende del dominio de aplicación)
 - Cantidad de registros eliminados.
 - Periodo de tiempo determinado.
 - Ante la necesidad de espacio.
 - Archivos muy volátiles → **no es conveniente**
 - Análisis de **recuperación** **dinámica** del **espacio** de almacenamiento.

Archivos

Eliminación

- Baja Lógica

- Se coloca una **marca especial** en los registros eliminados para hacer posible el posterior reconocimiento de los mismos.
- Permite **anular** el proceso de eliminación fácilmente → **costo de espacio**.
- Los programas que usan archivos deben cambiar su metodología de trabajo:
 - **LECTURA**: se debe ignorar registros eliminados.
 - **INSERCIÓN**: se debe reutilizar el espacio de registros eliminados.

- Recuperación dinámica de espacio
 - La recuperación dinámica de espacio (reasignación de espacio) consiste en **reutilizar** el espacio **al momento de insertar un nuevo registro** en el archivo.
 - Usando las marcas de borrado lógico se puede identificar un **espacio vacío** en el que se pueda **almacenar el nuevo registro**.

Archivos

Recuperación de espacio

- En archivos con registros de **longitud fija**
 - **Búsqueda secuencial**
 - Al insertar un nuevo registro se busca el **primer registro eliminado** (marcado).
 - Si no existe, se llega al final del archivo y se agrega allí.
 - Puede ser **muy lento** en algunos casos de aplicación.
 - Es necesario contar con un mecanismo que permita **saber de inmediato si existen espacios libres** en el archivo y **acceder directamente** a uno de ellos, en caso de existir.

Archivos

Recuperación de espacio

- En archivos con registros de longitud fija
 - **Lista encadenada o Pila**
 - Los espacios libres se mantienen **enlazados**.
 - Al insertar un nuevo registro → **cualquier espacio libre es bueno**.
 - La lista no necesita estar ordenada → **pila**.
 - Mínima reorganización al agregar o sacar elementos.

Archivos

Recuperación de espacio

- En archivos con registros de **longitud fija**
 - **Lista encadenada o Pila**
 - Para poder gestionar los espacios libres, se utiliza un **registro especial** ubicado en la **primera posición** del archivo.
 - Este registro se denomina **registro encabezado** y **no almacena información válida del dominio**, sino que se utiliza para administrar el archivo.
 - Se utiliza un campo del registro (generalmente el campo clave) para almacenar el enlace a otro registro.

- En archivos con registros de **longitud fija**
 - **Lista encadenada o Pila**
 - Se utiliza el **NRR** como **dirección de enlace** entre nodos.
 - El **NRR** que indica el primer registro disponible está en el **registro encabezado** del archivo.
 - En el caso de que no haya registros disponibles para ser reutilizados, se guarda el número **`-1`** en el registro encabezado.

Archivos

Recuperación de espacio

- Ejemplo de lista encadenada

Borrado NINGUNO	R0	R1	R2	R3	R4	R5	R6	R7	R8	R9
--------------------	----	----	----	----	----	----	----	----	----	----

↑
Registro Cabecera

Archivo Original

Borrado Posición 2	R0	R1	Borrado	R3	R4	R5	R6	R7	R8	R9
-----------------------	----	----	---------	----	----	----	----	----	----	----

↑
Registro Cabecera

Luego de borrar R2

Borrado Posición 5	R0	R1	Borrado	R3	R4	Próximo 2	R6	R7	R8	R9
-----------------------	----	----	---------	----	----	--------------	----	----	----	----

↑
Registro Cabecera

Luego de borrar R5

Archivos

Recuperación de espacio

- Ejemplo de lista encadenada



↑
Registro Cabecera

Luego de borrar R8



↑
Registro Cabecera

Luego de insertar R10

Archivos

Recuperación de espacio

- En archivos con registros de **longitud variable**
 - También se utiliza **la marca de borrado** para identificar a los registros eliminados.
 - El problema ahora es que el nuevo registro no se puede colocar en cualquier lugar → **debe caber**.
 - Se necesita realizar una **búsqueda** en la lista de lugares disponibles → **ya no se organiza como una pila**.

Archivos

Recuperación de espacio

- En archivos con registros de **longitud variable**
 - Los espacios libres se organizan mediante una **lista enlazada**.
 - No es posible utilizar el NRR como enlace, ya que los registros no tienen tamaño fijo.
 - En su lugar, se utiliza un **campo adicional** que **almacena explícitamente el enlace** (offset en bytes desde el inicio del archivo).
 - Cada registro almacena al inicio la **longitud efectiva en bytes**, lo que **permite determinar dónde comienza el siguiente**.

Archivos

Recuperación de espacio

- En archivos con registros de **longitud variable**
 - ¿Cómo se realiza la gestión del espacio libre?
 - Se busca un lugar disponible lo **suficientemente grande** para almacenar el nuevo registro.
 - Si el lugar encontrado **no coincide exactamente con el tamaño requerido**, se genera **fragmentación**.
 - Según cómo se asigne el espacio, la fragmentación puede ser:
 - **Interna**: queda el espacio sin utilizar dentro del bloque asignado al registro.
 - **Externa**: queda el espacio libre disperso en el archivo.

Archivos

Recuperación de espacio

- Fragmentación **interna**
 - Ocurre cuando se desperdicia espacio **dentro de un registro** → el lugar está asignado al registro pero éste no lo ocupa totalmente.
 - Puede darse en archivos con **reg. de longitud fija**.
 - Ej: no todos los nombres de personas ocupan la misma cantidad de caracteres.
 - Puede darse en archivos con **reg. de longitud variable**.
 - Al escribir por 1ra vez el archivo → NO
 - Al borrar un reg. y reemplazarlo por otro más corto → **SI**

- Fragmentación **externa**
 - Ocurre cuando el **espacio libre es demasiado pequeño** como para ser ocupado por un nuevo registro.
 - Soluciones
 - **Unir espacios libres pequeños adyacentes** para generar un espacio disponible mayor.
 - **Regenerar el archivo** → cuando hay mucha fragmentación.
 - **Minimizar la fragmentación**, eligiendo el espacio más “**adecuado**” en cada caso → **Estrategias de colocación**:
 - Primer ajuste.
 - Mejor ajuste.
 - Peor ajuste.

- Estrategias de colocación
 - **Primer ajuste**
 - Se selecciona la primera entrada de la lista de disponibles que pueda almacenar al registro, y **se le asigna de forma completa** al mismo.
 - Minimiza la búsqueda.
 - No se preocupa por la exactitud del ajuste.

Archivos

Recuperación de espacio

- Estrategias de colocación
 - **Mejor ajuste**
 - Elige la entrada que más se aproxime al tamaño del registro y **se le asigna de forma completa** al mismo.
 - Exige una búsqueda completa.

- Estrategias de colocación
 - **Peor ajuste**
 - Elige la entrada más grande para el registro, pero **sólo le asigna el espacio necesario** al mismo, quedando libre el resto.
 - El sobrante puede ser usado entonces por otro registro.
 - Exige una búsqueda completa.

- Estrategias de colocación
 - **Conclusiones**
 - Las estrategias de colocación sólo tienen sentido en archivos con registros de longitud variable.
 - De acuerdo a lo visto, en resumen:
 - **Primer ajuste:** más rápido, generalmente genera fragmentación interna.
 - **Mejor ajuste:** generalmente genera fragmentación interna.
 - **Peor ajuste:** potencialmente genera fragmentación externa.